

# The data verbs: `select` · `filter` · `mutate` · `arrange`

Tuesday, June 2, 2026

**i** Today: Tuesday June 2

## Before class

- Perusall Assignment: [DC §10.1–10.6](#)

## Plan for today

- Four new data verbs: `filter`, `arrange`, `select`, `mutate`
- Sorting verbs by *what they touch*: rows vs. columns
- Combining verbs with the pipe into real pipelines
- Formatting pipelines so they stay readable

## Helpful links

- [Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [R4DS Ch. 4](#) · [DC Ch. 10](#) · [dplyr reference](#)

## Recap from yesterday

We discussed the overarching **framework** and two data verbs:

| Family         | Input to Output          | The verb that uses it                  |
|----------------|--------------------------|----------------------------------------|
| Reduction      | variable to scalar       | <code>summarise()</code>               |
| Transformation | variable to variable     | <i>...today:</i> <code>mutate()</code> |
| Data verb      | data frame to data frame | all of them                            |

We write reduction functions inside of `summarise()`. Today we will cover `mutate()`, which contains **transformation** functions, plus three verbs that reshape *which* rows and columns you keep.

## A map of the verbs

We can organize dplyr by **what each verb operates on**:

Table 1: The dplyr toolkit.

| Touches...     | Verbs                                              | What changes                |
|----------------|----------------------------------------------------|-----------------------------|
| <b>Rows</b>    | <code>filter()</code> , <code>arrange()</code>     | which rows, and their order |
| <b>Columns</b> | <code>select()</code> , <code>mutate()</code>      | which columns, and new ones |
| <b>Groups</b>  | <code>group_by()</code> , <code>summarise()</code> | ( <i>yesterday</i> )        |

### Tip

These all follow the same three rules: **first argument is a data frame, you name columns without quotes, and the output is a brand-new data frame.**

## Rows: `filter()`

**`filter()`: keep the rows you want**

`filter()` keeps only the rows that pass a test (*DC* §10.2, *R4DS* §3.2.1):

```
penguins |>
  filter(species == "Gentoo")
```

```
# A tibble: 124 x 8
  species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
1 Gentoo  Biscoe           46.1           13.2           211           4500
2 Gentoo  Biscoe           50            16.3           230           5700
3 Gentoo  Biscoe           48.7           14.1           210           4450
4 Gentoo  Biscoe           50            15.2           218           5700
5 Gentoo  Biscoe           47.6           14.5           215           5400
6 Gentoo  Biscoe           46.5           13.5           210           4550
```

```

7 Gentoo Biscoe      45.4      14.6      211      4800
8 Gentoo Biscoe      46.7      15.3      219      5200
9 Gentoo Biscoe      43.3      13.4      209      4400
10 Gentoo Biscoe     46.8      15.4      215      5150
# i 114 more rows
# i 2 more variables: sex <fct>, year <int>

```

The test is a **transformation** that returns TRUE/FALSE for each row; `filter()` keeps the TRUEs.

## The comparison operators

These are the tests you'll use inside `filter()` (*DC* Table 10.9):

Table 2: Comparisons for `filter()`.

| Operator                | Means              | Example                                       |
|-------------------------|--------------------|-----------------------------------------------|
| <code>==</code>         | equal to           | <code>species == "Gentoo"</code>              |
| <code>!=</code>         | not equal to       | <code>sex != "male"</code>                    |
| <code>&gt; &gt;=</code> | greater (or equal) | <code>body_mass_g &gt;= 5000</code>           |
| <code>&lt; &lt;=</code> | less (or equal)    | <code>flipper_length_mm &lt; 190</code>       |
| <code>%in%</code>       | is one of          | <code>island %in% c("Dream", "Biscoe")</code> |

### Warning

Use `==` to **test** equality, not `=`. A single `=` means “assign,” not “test equality”.

## Multiple conditions

Separate tests with a comma (or `&`) for **AND**, a vertical bar `|` for **OR** (R4DS §3.2.1):

```

# AND: Adelies that are ALSO on Dream
penguins |>
  filter(species == "Adelie", island == "Dream")

# OR: any penguin that is Adelie OR Gentoo
penguins |>
  filter(species == "Adelie" | species == "Gentoo")

```

```
# %in% is the clean way to write a long OR
penguins |>
  filter(species %in% c("Adelie", "Gentoo"))
```

### **i** Note

`filter(species == "Adelie" | "Gentoo")` does **not** work the way it reads. Each side of `|` must be a complete test.

## Dropping NAs

Yesterday, `group_by(species, sex)` gave us an NA group because some penguins have a missing `sex`. Now we can drop them:

```
penguins |>
  filter(!is.na(sex)) |>
  group_by(species, sex) |>
  summarise(n = n(), .groups = "drop")
```

```
# A tibble: 6 x 3
  species    sex      n
  <fct>    <fct> <int>
1 Adelie  female   73
2 Adelie  male     73
3 Chinstrap female   34
4 Chinstrap male     34
5 Gentoo  female   58
6 Gentoo  male     61
```

`is.na(sex)` is TRUE for the missing ones; `!` flips it, so `!is.na(sex)` keeps the rows that **do** have a sex. This removes the NA group.

## Rows: `arrange()`

### `arrange()`: set the order

`arrange()` reorders rows without adding or removing any. This function gives the result in ascending order by default:

```
penguins |>
  arrange(body_mass_g)
```

```
# A tibble: 344 x 8
  species    island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
  <fct>      <fct>          <dbl>         <dbl>          <int>        <int>
1 Chinstrap Dream          46.9           16.6            192         2700
2 Adelie    Biscoe          36.5           16.6            181         2850
3 Adelie    Biscoe          36.4           17.1            184         2850
4 Adelie    Biscoe          34.5           18.1            187         2900
5 Adelie    Dream          33.1           16.1            178         2900
6 Adelie    Torgersen        38.6           17              188         2900
7 Chinstrap Dream          43.2           16.6            187         2900
8 Adelie    Biscoe          37.9           18.6            193         2925
9 Adelie    Dream          37.5           18.9            179         2975
10 Adelie   Dream          37            16.9            185         3000
# i 334 more rows
# i 2 more variables: sex <fct>, year <int>
```

You can wrap a column in `desc()` if you want descending order. If there are ties, to break them you can list several columns:

```
penguins |>
  arrange(species, desc(body_mass_g)) # by species, then heaviest first
```

## Columns: `select()`

**`select()`: keep the columns you want**

`select()` is to columns what `filter()` is to rows:

```
penguins |>
  select(species, island, body_mass_g)
```

```
# A tibble: 344 x 3
  species island    body_mass_g
  <fct>    <fct>        <int>
1 Adelie  Torgersen     3750
2 Adelie  Torgersen     3800
```

```

3 Adelie Torgersen      3250
4 Adelie Torgersen      NA
5 Adelie Torgersen      3450
6 Adelie Torgersen      3650
7 Adelie Torgersen      3625
8 Adelie Torgersen      4675
9 Adelie Torgersen      3475
10 Adelie Torgersen     4250
# i 334 more rows

```

You can grab a range, drop with `!`, or rename as you go:

```

penguins |> select(species:bill_depth_mm) # a range of columns
penguins |> select(!year)                 # everything EXCEPT year
penguins |> select(mass = body_mass_g)    # rename while selecting

```

### Note

`penguins` has only 8 columns, so `select()` feels optional here. On a 19-column table like `flights` from HW2, or a 76-column one, it's one of the first things you might want to try.

## `select()` helpers

For wide tables, name columns by pattern instead of one-by-one:

```

penguins |> select(starts_with("bill")) # bill_length_mm, bill_depth_mm
penguins |> select(ends_with("_mm"))    # all the millimetre measurements
penguins |> select(where(is.numeric))   # every numeric column

```

## Columns: `mutate()`

### `mutate()`: add a new column

`mutate()` adds columns computed from existing ones, keeping every row:

```

penguins |>
  mutate(body_mass_kg = body_mass_g / 1000) |>
  select(species, body_mass_g, body_mass_kg)

```

```
# A tibble: 344 x 3
  species body_mass_g body_mass_kg
  <fct>      <int>      <dbl>
1 Adelie    3750        3.75
2 Adelie    3800        3.8
3 Adelie    3250        3.25
4 Adelie     NA        NA
5 Adelie    3450        3.45
6 Adelie    3650        3.65
7 Adelie    3625        3.62
8 Adelie    4675        4.68
9 Adelie    3475        3.48
10 Adelie   4250        4.25
# i 334 more rows
```

## mutate() contains transformation functions

`mutate()` **always** involves a *transformation* function, just as `summarise()` **always** involves a *reduction* function.

```
# transformation: one value per row to mutate()
penguins |> mutate(bill_ratio = bill_length_mm / bill_depth_mm)

# reduction: one value total to summarise()
penguins |> summarise(avg_bill = mean(bill_length_mm, na.rm = TRUE))
```

### Tip

Deciding between them is the same question from yesterday: does the calculation give **one value per row** (`mutate`) or **one value for the whole group** (`summarise`)?

## A categorical column with ifelse()

`ifelse()` is a transformation that picks between two values per row, perfect inside `mutate()`:

```
penguins |>
  mutate(size = ifelse(body_mass_g > 4000, "large", "small")) |>
  select(species, body_mass_g, size)
```

```
# A tibble: 344 x 3
  species body_mass_g size
  <fct>      <int> <chr>
1 Adelie      3750 small
2 Adelie      3800 small
3 Adelie      3250 small
4 Adelie       NA <NA>
5 Adelie      3450 small
6 Adelie      3650 small
7 Adelie      3625 small
8 Adelie      4675 large
9 Adelie      3475 small
10 Adelie      4250 large
# i 334 more rows
```

## Combining verbs

### The pipe does the real work

One verb does one thing. For most questions, we will need several, chained together with `|>`:

```
penguins |>
  filter(!is.na(sex)) |>
  mutate(body_mass_kg = body_mass_g / 1000) |>
  select(species, sex, body_mass_kg) |>
  arrange(desc(body_mass_kg))
```

This can be read in English as: take penguins, **then** drop missing sex, **then** add mass in kg, **then** keep three columns, **then** sort heaviest first.

#### Tip

Because each verb returns a new data frame, the output of one is the input to the next. The verbs line up on the left so you can quickly skim to understand what's going on.

### Putting all six together

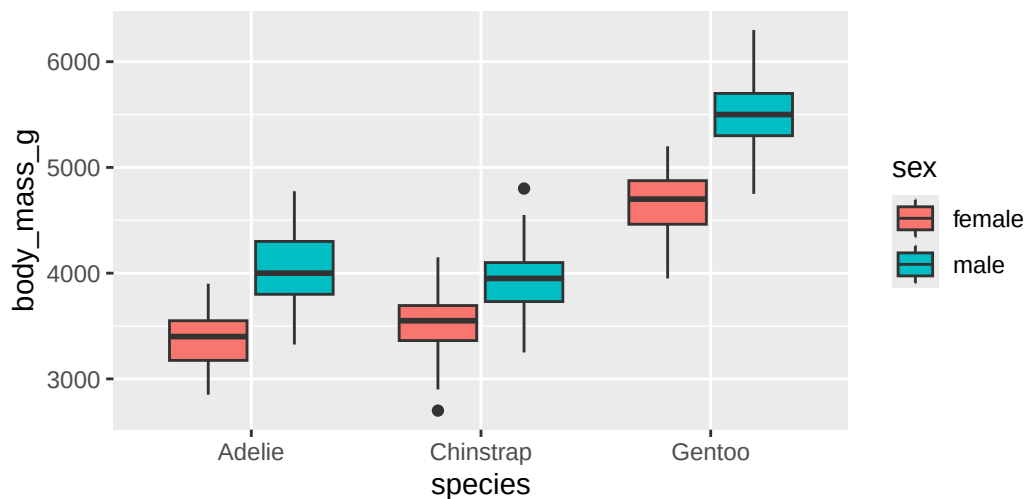
We can use all six verbs to answer the question, “what’s the average mass of each species/sex, biggest first?”:

```
penguins |>
  filter(!is.na(sex)) |>
  group_by(species, sex) |>
  summarise(
    n      = n(),
    avg_mass = mean(body_mass_g, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(avg_mass))
```

```
# A tibble: 6 x 4
  species sex      n avg_mass
  <fct>   <fct> <int>   <dbl>
1 Gentoo  male     61   5485.
2 Gentoo  female   58   4680.
3 Adelie  male     73   4043.
4 Chinstrap male    34   3939.
5 Chinstrap female   34   3527.
6 Adelie  female   73   3369.
```

And because the result is just a data frame, you can pipe it straight into a plot (switching `|>` to `+`):

```
penguins |>
  filter(!is.na(sex)) |>
  ggplot(aes(x = species, y = body_mass_g, fill = sex)) +
  geom_boxplot()
```



## Formatting your pipelines

A multi-step pipeline is only readable if you format it. The rules that matter most:

- A space before `|>`, and let `|>` be the **last thing on the line**.
- For verbs with **named arguments** (`mutate`, `summarise`), put each argument on its **own line**, indented two spaces.
- Verbs without named arguments (`filter`, `select`, `arrange`) can stay on one line if they fit.
- Name new columns in `snake_case`: `body_mass_kg`, not `BodyMassKG`.

|                                                                                                                                                                         |                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <pre># Strive for penguins  &gt;   filter(!is.na(sex))  &gt;   group_by(species)  &gt;   summarise(     n    = n(),     avg = mean(body_mass_g, na.rm = TRUE)   )</pre> | <pre># Avoid penguins &gt;filter(!is.na(sex)) &gt;group_by( species) &gt;summarize(n=n(),avg=mean( body_mass_g,na.rm=TRUE))</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|

### Tip

Don't hand-format forever: the **styler** package reformats a whole file for you. In RStudio, Cmd/Ctrl + Shift + P then type “styler”. Full guide: R4DS Ch. 4.

## A helpful shortcut

Counting rows per group is so common it has its own verb, `count()`:

```
penguins |> count(species)

# is exactly the same as
penguins |>
  group_by(species) |>
  summarise(n = n())
```

### Note

`count()` is just `group_by()` + `summarise(n = n())` bundled up. Add `sort = TRUE` to get the biggest groups first.

## What we covered today

- **filter()**: keep rows that pass a test; `==` not `=`; combine with `,/|/%in%`; `!is.na()` to drop missing
- **arrange()**: reorder rows; **desc()** for descending; extra columns break ties
- **select()**: keep/drop/rename columns; **starts\_with()** & friends for wide tables
- **mutate()**: add columns from transformations; the per-row twin of **summarise()**
- **The pipe**: chain verbs into a readable top-to-bottom story; pipe results into ggplot
- **Style**: space before `|>`, named args on their own lines, **snake\_case**, let **styler** do the rest

## Activity

### Activity: wrangling the penguins

Roughly 25 minutes. Individually or with a neighbor.

- [Activity Canvas Link](#)

**Format:** a handout that walks each verb, then asks you to chain them. You'll match tasks to verbs, write **filter/arrange/select/mutate** pipelines on **penguins**, debug a `=` vs `==` error, build one multi-verb pipeline that answers a question, and restyle a deliberately unreadable pipeline.

**To turn in:** save your `.qmd` and rendered HTML.

## Wrap-up & looking ahead

**Tomorrow (Wednesday):** no new reading, we put all six verbs together on a messier dataset and turn raw rows into summary tables and plots.

### Upcoming Deadlines

- **Thu 6/4, before class:** [DC §12.1](#)
- **Thu 6/4, 11:59 p.m.:** [Project: Team formation](#)
- **Fri 6/5, 11:59 p.m.:** [Homework 3: Data Wrangling & Reshaping](#)

### Reach out

Stuck? Office hours, or email me ([cgc5478@psu.edu](mailto:cgc5478@psu.edu)) or Xinyue ([xpw5228@psu.edu](mailto:xpw5228@psu.edu)).

### Reference

[Syllabus](#) · [AI policy](#) · [Schedule](#) · [R4DS Ch. 3](#) · [R4DS Ch. 4](#) · [DC Ch. 10](#) · [dplyr reference](#)